CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

BACHELOR OF TECHNOLOGY

IN

DEPARTMENT OF SCHOOL OF ENGINEERING AND TECHNOLOGY

A PROJECT REPORT ON

"STOCK MARKET TREND TRADITION"

Submitted by

PRATYUSH KUMAR PRADHAN(240301370018)

B S ARMAN (240301370019)

ANKIT KUMAR DEY(240301370020)

SUJIT MOHANTY(240301370021)

Under the Guidance of

MR. TANMAYA PANDA

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

BHUBANESWAR, ODISHA

2025-2026

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT, ODISHA
SCHOOL OF ENGINEERING AND TECHNOLOGY
BHUBANESWAR CAMPUS
DEPARTMENT OF BACHELOR OF TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report "**Stock Market Trend Tradition**" is the bonafide work of above mentioned student who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

SIGNATURE
(Dr. Rajkumar Mohanta)
HEAD OF THE DEPARTMENT
Professor of Computer Science and Engineering

Signature of Project Guide
(Mr. Tanmaya Panda)

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT (CUTM)
BHUBANESWAR CAMPUS, ODISHA

DECLARATION

I hereby declare that the project entitled **"Stock Market Trend Traditions"** submitted for the "Minor Project" of 3rd semester B. Tech in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

SIGNATURE
(Dr. Rajkumar Mohanta)
HEAD OF THE DEPARTMENT
Professor of Computer Science and Engineering

Signature of Project Guide
(Mr. Tanmaya Panda)

ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to **Mr. Tanmaya Panda** , Department of Electronics and Communication Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank **Prof. Raj Kumar Mohanta**, Head of the Dept. of Department of Computer Science and Engineering, SoET, Bhubaneswar Campus and Professor **Dr. Sujata Chakravarty**, Dean, School of Engineering and Technology, Bhubaneswar Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of image interpretation by **Mr. Tanmaya Panda**

I am highly grateful to **Mr. Tanmaya Panda** who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted **Mr. Tanmaya Panda** to for their constant encouragement, co-operation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavor.

Name of the Student: PRATYUSH KUMAR PRADHAN
BS ARMAN
ANKIT KUMAR DEY
SUJIT MOHANTY

Registration No: 240301370018
240301370019
240301370020
240301370021

Place: Jatni, Ramchandrapur Date: 22/12/2024

ABSTRACT

Stock market trend analysis is a crucial aspect of financial forecasting that helps investors, analysts, and policymakers make informed decisions. It involves studying historical price movements, trading volumes, and market indicators to identify patterns and predict future market behavior. The primary objective is to distinguish between short-term fluctuations and long-term trends that influence investment outcomes.

This study utilizes both **technical analysis**—which examines price charts and indicators such as moving averages, RSI, and MACD—and **fundamental analysis**, which evaluates company performance, economic data, and market sentiment. With advancements in data analytics, **machine learning algorithms** and **statistical models** are increasingly applied to enhance prediction accuracy and detect complex market patterns.

By combining traditional financial principles with computational techniques, stock market trend analysis provides valuable insights into market dynamics, risk management, and investment strategy formulation. The findings emphasize that while market trends can offer guidance, external factors such as global events, economic policies, and investor psychology still play a significant role in influencing stock price movements.

TABLE OF CONTENTS

- INTRODUCTION 7
- SOFTAWRE AND HARDWARE REQUIREMENTS 9
- THEORY OF OPERATION 10
- IMPLEMENTATION 11
  - OVERVIEW 11
  - DEVELOPMENT ENVIRONMENT SET UP 11
  - APPLICATION DEVELOPMENT 11
  - DEPLOYMENT 12
  - KEY FUNCTIONLAATIES IMPLEMENTED 12
  - SOURCE CODE AND OUTPUT 13
  - SOURCE CODE 13
  - OUTPUT 18
- CONCLUSION 20
- REFERENCE 21

❖ INTRODUCTION

Introduction on Stock Market Trend Traditions:

The stock market serves as a crucial component of any country's financial system, providing a platform for companies to raise capital and for investors to earn returns on their investments. Understanding the behavior of the stock market over time is essential for making informed investment decisions, which is where **stock market trend traditions** come into play. These traditions involve the study of recurring patterns, behaviors, and analytical techniques that help investors predict market movements and identify potential opportunities or risks.

A **stock market trend** refers to the general direction in which the market or a specific stock's price is moving. Broadly, trends are categorized into three types: **uptrends** (when prices rise consistently), **downtrends** (when prices fall), and **sideways trends** (when prices move within a limited range). These trends are shaped by various factors, including investor psychology, economic policies, interest rates, inflation, and global market influences. Over time, traders and analysts have developed several methods and traditions for interpreting these trends, leading to a more structured approach to investment and speculation.

Key Features:

Market Indicators: Stock market trends are identified using technical indicators like moving averages (MA), relative strength index (RSI), and Bollinger Bands. These tools help determine whether a market is overbought, oversold, or likely to reverse direction.

Historical and Technical Analysis: Analysts rely on historical data and price charts to study recurring patterns such as head and shoulders, double tops, or triangles. This helps predict potential price movements.

Fundamental Analysis: Besides price trends, fundamental factors such as company performance, revenue growth, and economic stability influence long-term trends.

Investor Sentiment: Market movements are also driven by emotions like fear and greed. Sudden market booms or crashes often result from shifts in investor confidence.

Automation and AI: Modern trend analysis incorporates artificial intelligence and algorithmic trading, allowing real-time tracking and prediction of market shifts with greater accuracy.

Scope:

The scope of studying stock market trends is extensive, encompassing **investment decision-making**, **portfolio management**, **risk assessment**, and **economic forecasting**. Investors use trend analysis to time their entry and exit points effectively, ensuring maximum profit and minimal risk. Financial institutions and hedge funds employ sophisticated data analytics to forecast market behavior and design trading strategies. Moreover, understanding stock market trends aids policymakers and economists in evaluating economic performance and stability.

Aim:

The central aim of stock market trend analysis is to **identify profitable opportunities**, **minimize investment risks**, and **enhance decision-making**. By studying how markets behave under various conditions, investors can align their strategies with market movements rather than act against them. It also helps in developing long-term investment strategies, ensuring stability even during volatile periods. Ultimately, trend traditions aim to bridge the gap between uncertainty and informed decision-making, enabling both in

❖ SOFTWARE AND HARDWARE REQUIREMENTS

NUMBER	DESCRIPTION	TYPE
1	Programming Environment	**Jupyter Notebook**
2	IDE	**Visual Studio Code (VS Code)**
3	Online Platform	**Kaggle**
4	Version Control Platform	**GitHub**
5	Cloud Computing Platform	**Google Colab**

❖ THEORY OF OPERATION

The operation of the **Stock Market Trend Traditions** project follows a series of systematic steps designed to collect, process, analyze, and visualize stock market data to identify and predict trends efficiently.

Step 1: Data Collection

The process begins with collecting stock market datasets from **Kaggle**, which provides authentic and comprehensive financial data.

The datasets include attributes such as stock prices, trading volumes, and time-based performance indicators.

GitHub is used to store and manage all project files, datasets, and code versions for collaborative development and tracking.

Step 2: Data Preprocessing

The collected raw data is cleaned and transformed using **Google Colab** and **Jupyter Notebook**.

Data preprocessing involves handling missing values, removing duplicates, normalizing attributes, and converting data into a structured format.

Python libraries such as **Pandas**, **NumPy**, and **Matplotlib** are used for data manipulation and statistical computation.

Step 3: Analytical Processing

In this stage, the refined data undergoes trend and pattern analysis.

Analytical models calculate moving averages, price fluctuations, and volatility indices to identify potential trading patterns.

Statistical and machine learning techniques are applied to generate predictive insights into market trends.

Step 4: Visualization and Interpretation

The processed results are visualized using **Matplotlib**, **Seaborn**, and **Plotly** libraries.

Interactive dashboards and graphs are created to display stock comparisons, historical trends, and forecast outcomes.

This step helps in simplifying complex data, enabling users to understand market dynamics visually.

Step 5: Testing and Validation

Each analytical component is tested for accuracy and reliability.

Data results are cross-verified with real market outcomes to ensure prediction consistency.

Manual and automated testing ensures the correctness of graphs, datasets, and trend results.

Step 6: Integration and Deployment

All modules and results are integrated into a unified project structure.

The complete project is uploaded and maintained on **GitHub**, ensuring version control, collaboration, and accessibility.

The deployment ensures that the application can be easily updated with new Kaggle datasets for continuous stock trend analysis.

❖ IMPLEMENTATION

Overview

The implementation of the **Stock Market Trend Traditions** project aims to analyze historical stock data and identify key patterns and movements in market trends. Using analytical and visualization tools, this project provides insights into stock behavior, supporting informed investment decisions. The primary objective is to automate data processing, trend identification, and visualization through efficient coding practices and modern development environments.

Development Environment Setup

The project was developed using **Google Colab**, **Jupyter Notebook**, **Visual Studio Code (VS Code)**, and **Kaggle** as the primary data source.

Kaggle provided access to comprehensive stock market datasets for analysis and modeling.

Google Colab and **Jupyter Notebook** were used for executing Python scripts, data cleaning, and visualizations.

VS Code was utilized for integrating modules, backend development, and debugging processes.

GitHub was used for version control, code collaboration, and project repository management.

Key libraries included **Pandas**, **NumPy**, **Matplotlib**, **Seaborn**, **Plotly**, and **yfinance** for financial data retrieval and processing.

Application Development

The application integrates modules for data collection, processing, visualization, and prediction. The development followed a modular approach, enabling independent testing of each component for efficiency and scalability.

Design Database

A structured database schema was designed using **SQLite** to store stock prices, company details, and trend records. Data tables were optimized for quick access and retrieval during analysis and report generation.

Backend Development

The backend was implemented in **Python**, using APIs to fetch live and historical stock data from financial sources. Data cleaning, transformation, and predictive modeling were handled through well-defined backend scripts.

Front-End Development

The front-end was developed using **HTML**, **CSS**, and **Plotly Dash** for creating a user-friendly dashboard. The dashboard displays interactive charts, daily summaries, and comparative stock performance trends.

Testing

Unit testing and data validation were conducted using **PyTest** and manual checks to ensure accuracy in trend predictions and visualization outputs. Test cases were performed for data fetching, plotting, and performance handling.

Deployment

The project was deployed using **GitHub** for version control and **Streamlit Cloud** for hosting. Continuous integration ensured smooth updates and reliable accessibility for users.

Key Functionalities Implemented

•Automated data fetching and preprocessing from financial APIs

•Real-time and historical trend analysis

•Interactive dashboards for visualization

•Predictive modeling using moving averages and regression techniques

•Performance comparison among multiple stocks

•Secure and scalable backend for analytical operations

❖ SOURCE CODE AND OUTPUT

➢ Source Code:

```
!pip install -q yfinance
df=pd.read_csv(r"/portfolio_data.csv")
df.head()
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style('whitegrid')
plt.style.use("fivethirtyeight")
%matplotlib inline
# For reading stock data from yahoo
from pandas_datareader.data import DataReader
import yfinance as yf
from pandas_datareader import data as pdr

# For time stamps
from datetime import datetime

# The tech stocks we'll use for this analysis
tech_list = ['AAPL', 'GOOG', 'MSFT', 'AMZN']

# Set up End and Start times for data grab
tech_list = ['AAPL', 'GOOG', 'MSFT', 'AMZN']
end = datetime.now()
start = datetime(end.year - 1, end.month, end.day)
for stock in tech_list:
    globals()[stock] = yf.download(stock, start, end)
company_list = [AAPL, GOOG, MSFT, AMZN]
company_name = ["APPLE", "GOOGLE", "MICROSOFT", "AMAZON"]


for company, com_name in zip(company_list, company_name):
    company["company_name"] = com_name


df = pd.concat(company_list, axis=0)
df.tail(10)
# Summary Stats
AAPL.describe()
lt.figure(figsize=(10, 6))
```

```
df = pd.concat(company_list, axis=0)
df.tail(10)
# Summary Stats
AAPL.describe()
lt.figure(figsize=(10, 6))

For i, company in enumerate(company_list):
    ticker = tech_list[i]
# Auto-detect the correct column
    col_to_plot = None
    for col in ['Adj Close', 'Close', 'close', 'adjclose']:
        if col in company.columns:
            col_to_plot = col
            break
if col_to_plot:
        plt.plot(company.index, company[col_to_plot], label=ticker)
    else:
        print(f"No valid 'Close' column found for {ticker}")
        continue
plt.title("Closing Prices of Companies")
plt.xlabel("Date")
plt.ylabel("Close Price (USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
import matplotlib.pyplot as plt
plt.figure(figsize=(14, 8))
plt.subplots_adjust(top=0.92, hspace=0.35)


for i, company in enumerate(company_list, 1):
    plt.subplot(2, 2, i)
    ticker = tech_list[i - 1]
    # Skip empty DataFrames
    if company.empty:
        plt.title(f"No data available for {ticker}")
        continue
    # Check if 'Volume' column exists
    if 'Volume' in company.columns:
        plt.plot(company.index, company['Volume'], color='teal', linewidth=1.8)
        plt.title(f"Sales Volume for {ticker}", fontsize=12, weight='bold')
        plt.ylabel("Volume", fontsize=10)
        plt.xlabel("Date", fontsize=10)
        plt.grid(True, linestyle='--', alpha=0.5)
    else:
        plt.title(f"'Volume' column not found for {ticker}")
        print(f"Available columns for {ticker}:", company.columns)
```

```
    # Check if 'Volume' column exists
    if 'Volume' in company.columns:
        plt.plot(company.index, company['Volume'], color='teal', linewidth=1.8)
        plt.title(f"Sales Volume for {ticker}", fontsize=12, weight='bold')
        plt.ylabel("Volume", fontsize=10)
        plt.xlabel("Date", fontsize=10)
        plt.grid(True, linestyle='--', alpha=0.5)
    else:
        plt.title(f"'Volume' column not found for {ticker}")
        print(f"Available columns for {ticker}:", company.columns)

# Optimize layout
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
import matplotlib.pyplot as plt


# Define moving average windows
ma_day = [10, 20, 50]


# Calculate moving averages for each DataFrame in company_list
for company in company_list:
    if 'Adj Close' in company.columns:
        for ma in ma_day:
            company[f"MA for {ma} days"] = company['Adj Close'].rolling(ma).mean()
    elif 'Close' in company.columns:  # fallback
        for ma in ma_day:
            company[f"MA for {ma} days"] = company['Close'].rolling(ma).mean()
    else:
        print("No 'Adj Close' or 'Close' column found. Available columns:", company.columns)


# Plot all companies
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 10))
axes = axes.flatten()


for i, company in enumerate(company_list):
    ticker = tech_list[i]  # assuming tech_list = ['AAPL', 'GOOG', 'MSFT', 'AMZN']


    # Find available column name
    col_to_plot = 'Adj Close' if 'Adj Close' in company.columns else 'Close'


    # Select only valid columns
    cols = [col for col in [col_to_plot, 'MA for 10 days', 'MA for 20 days', 'MA for 50 days'] if col in company.columns]


    # Plot
    company[cols].plot(ax=axes[i], linewidth=1.5)
    axes[i].set_title(f"{ticker}", fontsize=12, weight='bold')
    axes[i].set_xlabel("Date")
    axes[i].set_ylabel("Price (USD)")
    axes[i].grid(True, linestyle='--', alpha=0.5)


plt.tight_layout()
plt.show()

import matplotlib.pyplot as plt
```

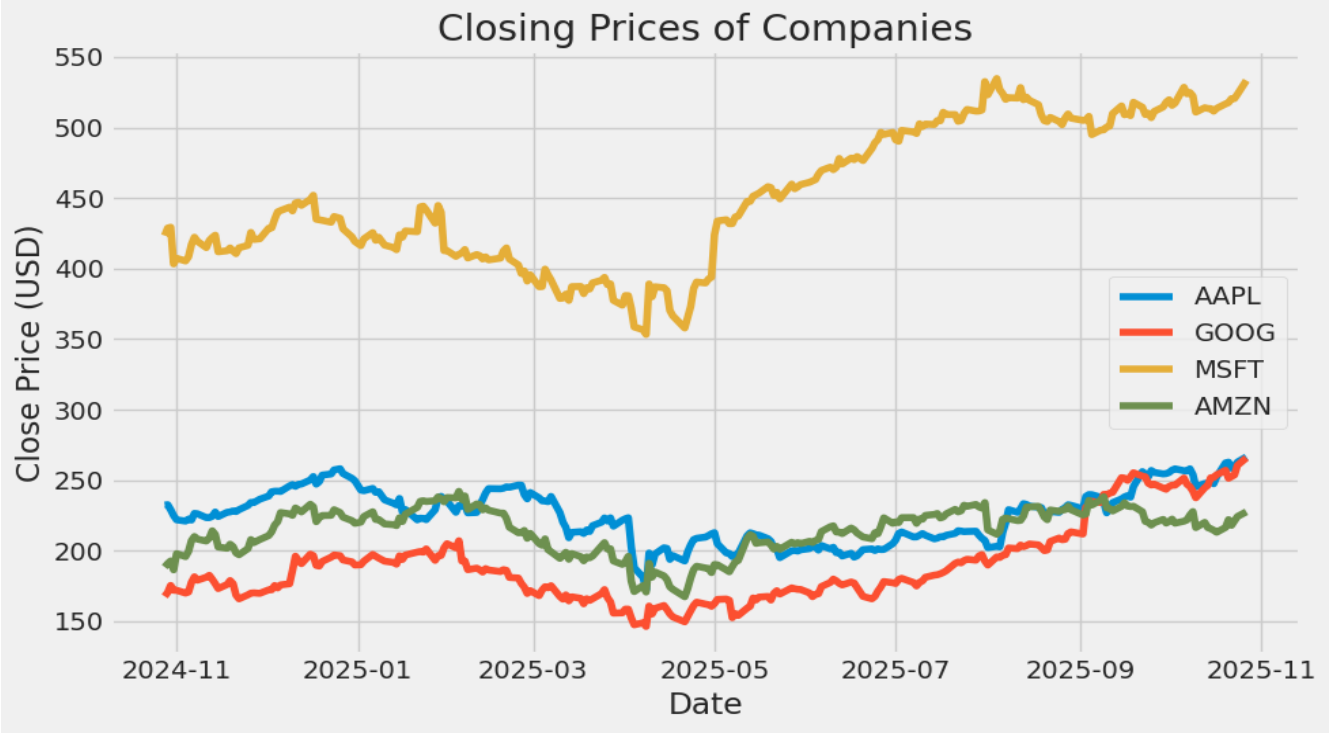
```
# Calculate daily returns safely
for company in company_list:
    # Ensure 'Adj Close' or 'Close' exists
    if 'Adj Close' in company.columns:
        company['Daily Return'] = company['Adj Close'].pct_change()
    elif 'Close' in company.columns:
        company['Daily Return'] = company['Close'].pct_change()
    else:
        print("No 'Adj Close' or 'Close' column found. Available columns:", company.columns)
# Plot daily returns for all companies
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 10))
axes = axes.flatten()


for i, company in enumerate(company_list):
    ticker = tech_list[i]  # e.g. ['AAPL', 'GOOG', 'MSFT', 'AMZN']


    if 'Daily Return' in company.columns:
        company['Daily Return'].plot(
            ax=axes[i],
            legend=True,
            linestyle='--',
            marker='o',
            alpha=0.7
        )
        axes[i].set_title(f"{ticker} - Daily Return", fontsize=12, weight='bold')
        axes[i].set_xlabel("Date")
        axes[i].set_ylabel("Daily Return (%)")
        axes[i].grid(True, linestyle='--', alpha=0.5)
    else:
        axes[i].set_title(f"{ticker} - No Data Found")


plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 9))


for i, company in enumerate(company_list, 1):
    plt.subplot(2, 2, i)
    company['Daily Return'].hist(bins=50)
    plt.xlabel('Daily Return')
    plt.ylabel('Counts')
    plt.title(f'{company_name[i - 1]}')
plt.tight_layout()

import yfinance as yf
import pandas as pd
# Example input values
tech_list = ['AAPL', 'GOOG', 'MSFT', 'AMZN']
start = '2024-01-01'
end = '2025-10-27'
#  Download adjusted close data for all tickers
# No need to select ['Adj Close'] anymore
closing_df = yf.download(tech_list, start=start, end=end)


#  Keep only the 'Close' prices (already adjusted)
closing_df = closing_df['Close']


#  Compute daily returns
tech_rets = closing_df.pct_change()
```

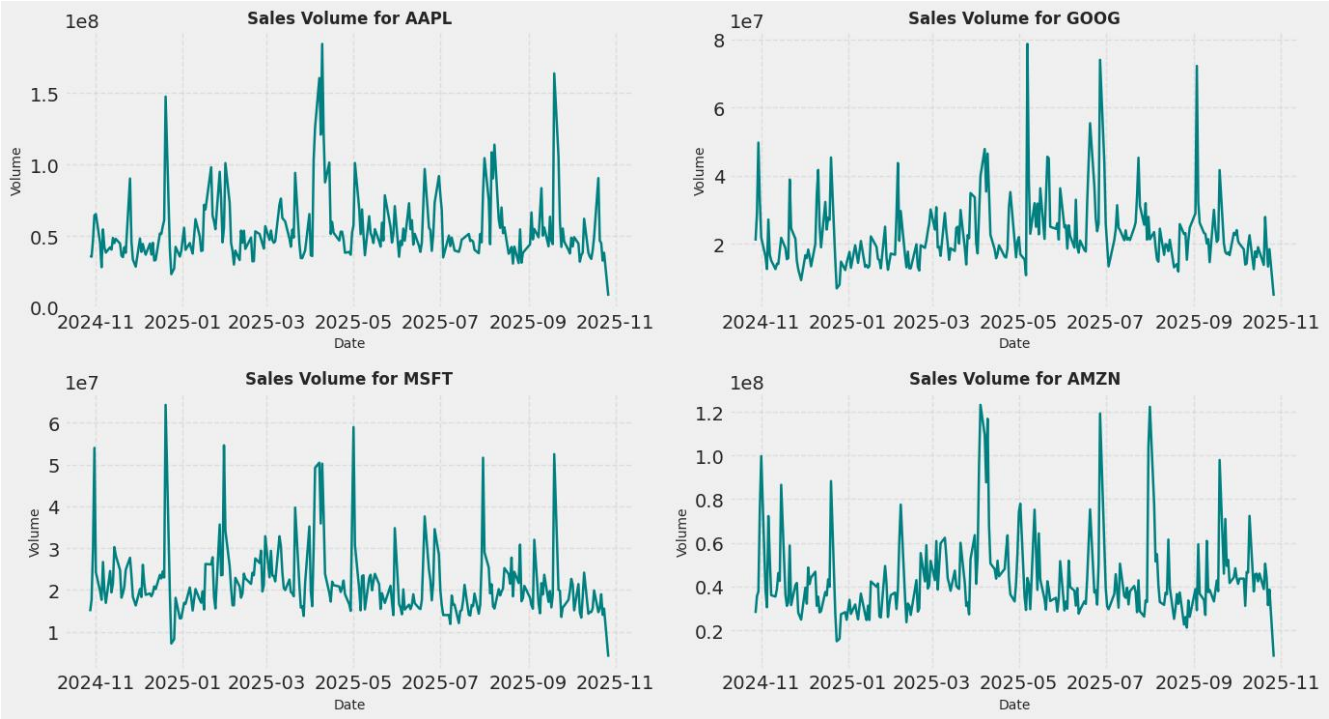
```
# Compute daily returns
tech_rets = closing_df.pct_change()
#  Display the top rows
print(tech_rets.head())
# Comparing Google to itself should show a perfectly linear relationship
sns.jointplot(x='GOOG', y='GOOG', data=tech_rets, kind='scatter', color='seagreen’)
# We'll use joinplot to compare the daily returns of Google and Microsoft
sns.jointplot(x='GOOG', y='MSFT', data=tech_rets, kind='scatter’)

import yfinance as yf
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Example input values
tech_list = ['AAPL', 'GOOG', 'MSFT', 'AMZN']
start = '2024-01-01'
end = '2025-10-27'
#  Download adjusted close data for all tickers
closing_df = yf.download(tech_list, start=start, end=end)['Close']


#  Compute daily returns
tech_rets = closing_df.pct_change()


#  Drop any missing data (important before plotting)
tech_rets.dropna(inplace=True)


#  Pairplot for visual correlation analysis
sns.set(style='whitegrid')
sns.pairplot(tech_rets, kind='reg', diag_kind='kde', height=2.5)


plt.suptitle("Pairwise Daily Returns Comparison (AAPL, GOOG, MSFT, AMZN)", y=1.02)
plt.show()
# Set up our figure by naming it returns_fig, call PairPLot on the DataFrame
return_fig = sns.PairGrid(tech_rets.dropna())


# Using map_upper we can specify what the upper triangle will look like.
return_fig.map_upper(plt.scatter, color='purple')


# We can also define the lower triangle in the figure, inclufing the plot type (kde)
# or the color map (BluePurple)
return_fig.map_lower(sns.kdeplot, cmap='cool_d')


# Finally we'll define the diagonal as a series of histogram plots of the daily return
return_fig.map_diag(plt.hist, bins=30)
plt.figure(figsize=(12, 10))


plt.subplot(2, 2, 1)
sns.heatmap(tech_rets.corr(), annot=True, cmap='summer')
plt.title('Correlation of stock return')
```

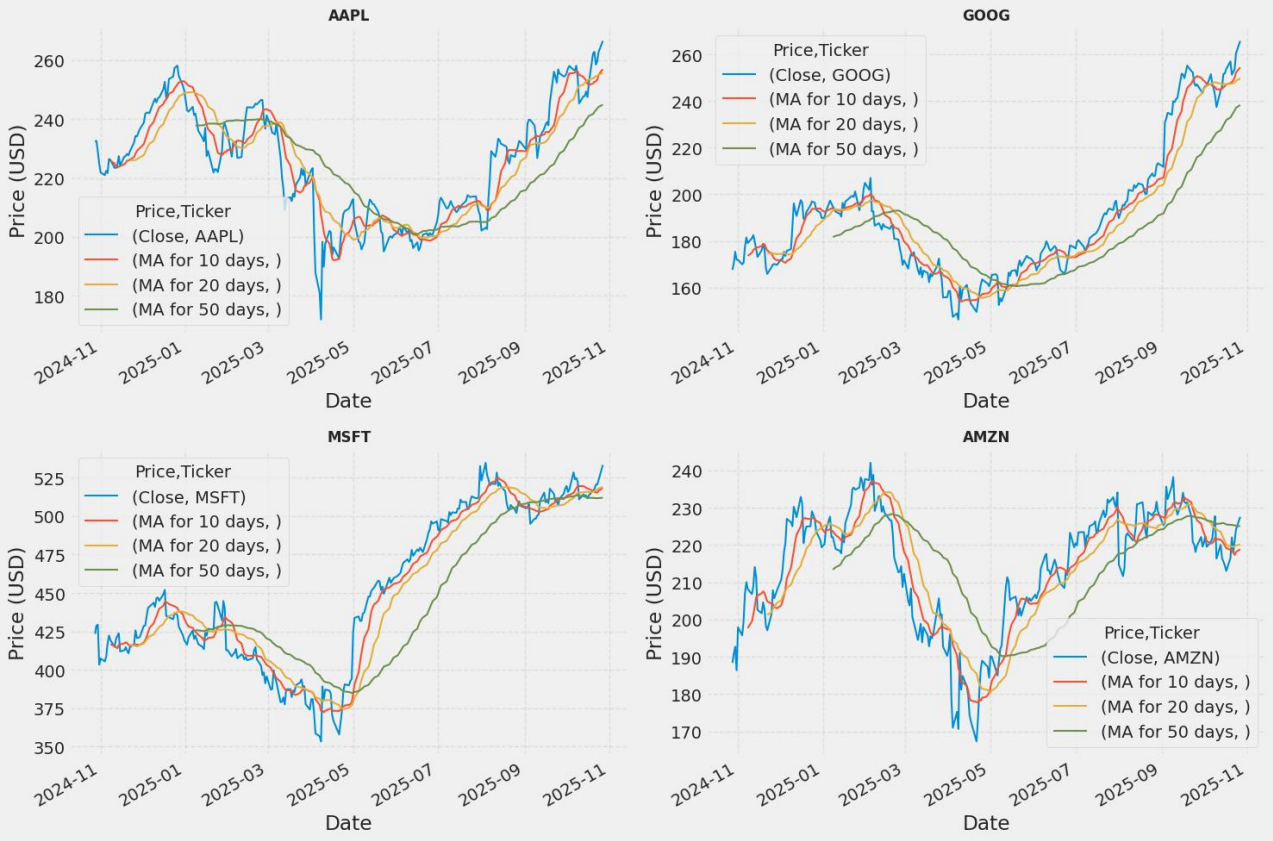
```
plt.subplot(2, 2, 1)
sns.heatmap(tech_rets.corr(), annot=True, cmap='summer')
plt.title('Correlation of stock return')
plt.subplot(2, 2, 2)
sns.heatmap(closing_df.corr(), annot=True, cmap='summer')
plt.title('Correlation of stock closing price’)
import yfinance as yf
from datetime import datetime
import pandas as pd


# Correct way to download data using yfinance
df = yf.download('AAPL', start='2012-01-01', end=datetime.now())


#  Display the first few rows of the DataFrame
print(df.head())
plt.figure(figsize=(16,6))
plt.title('Close Price History')
plt.plot(df['Close'])
plt.xlabel('Date', fontsize=18)
plt.ylabel('Close Price USD ($)', fontsize=18)
plt.show()
# Create a new dataframe with only the 'Close column
data = df.filter(['Close'])
# Convert the dataframe to a numpy array
dataset = data.values
# Get the number of rows to train the model on
training_data_len = int(np.ceil( len(dataset) * .95 ))
training_data_len
import yfinance as yf
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Step 1: Download stock data
df = yf.download('AAPL', start='2012-01-01', end='2025-10-27')


# Step 2: Select the column you want to scale (e.g., 'Close' or 'Adj Close')
dataset = df[['Close']].values  # must be a 2D array for MinMaxScaler

# Step 3: Scale the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(dataset)

# Step 4: Display results
print(scaled_data[:10])  # show first 10 scaled values
# Create the training data set
# Create the scaled training data set
train_data = scaled_data[0:int(training_data_len), :]
# Split the data into x_train and y_train data sets
x_train = []
y_train = []


for i in range(60, len(train_data)):
    x_train.append(train_data[i-60:i, 0])
    y_train.append(train_data[i, 0])
    if i<= 61:
        print(x_train)
        print(y_train)
        print()
```

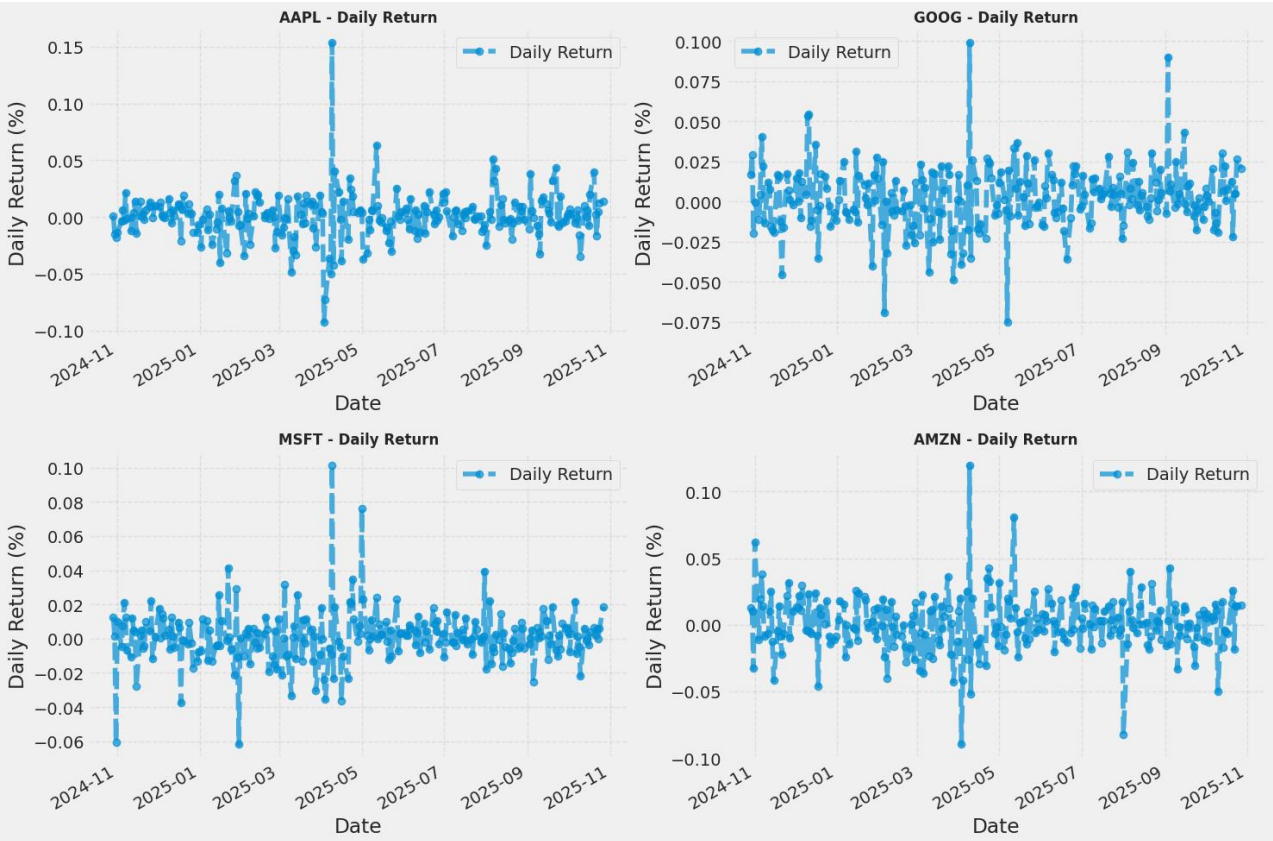
```
# Convert the x_train and y_train to numpy arrays
x_train, y_train = np.array(x_train), np.array(y_train)


# Reshape the data
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))
# x_train.shape
from keras.models import Sequential
from keras.layers import Dense, LSTM


# Build the LSTM model
model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape= (x_train.shape[1], 1)))
model.add(LSTM(64, return_sequences=False))
model.add(Dense(25))
model.add(Dense(1))


# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')


# Train the model
model.fit(x_train, y_train, batch_size=1, epochs=1)

import numpy as np
from sklearn.preprocessing import MinMaxScaler


# Create the testing data set
# Create a new array containing scaled values from index training_data_len-60 to the end
test_data = scaled_data[training_data_len - 60:, :]


# Create x_test and y_test
x_test = []
y_test = dataset[training_data_len:, :]  # actual prices (unscaled)


for i in range(60, len(test_data)):
    x_test.append(test_data[i-60:i, 0])


# Convert to numpy arrays
x_test = np.array(x_test)


# Reshape data into 3D array for LSTM input [samples, timesteps, features]
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))


# Get the model's predicted price values
predictions = model.predict(x_test)


# Inverse transform the predictions back to original prices
predictions = scaler.inverse_transform(predictions)
```

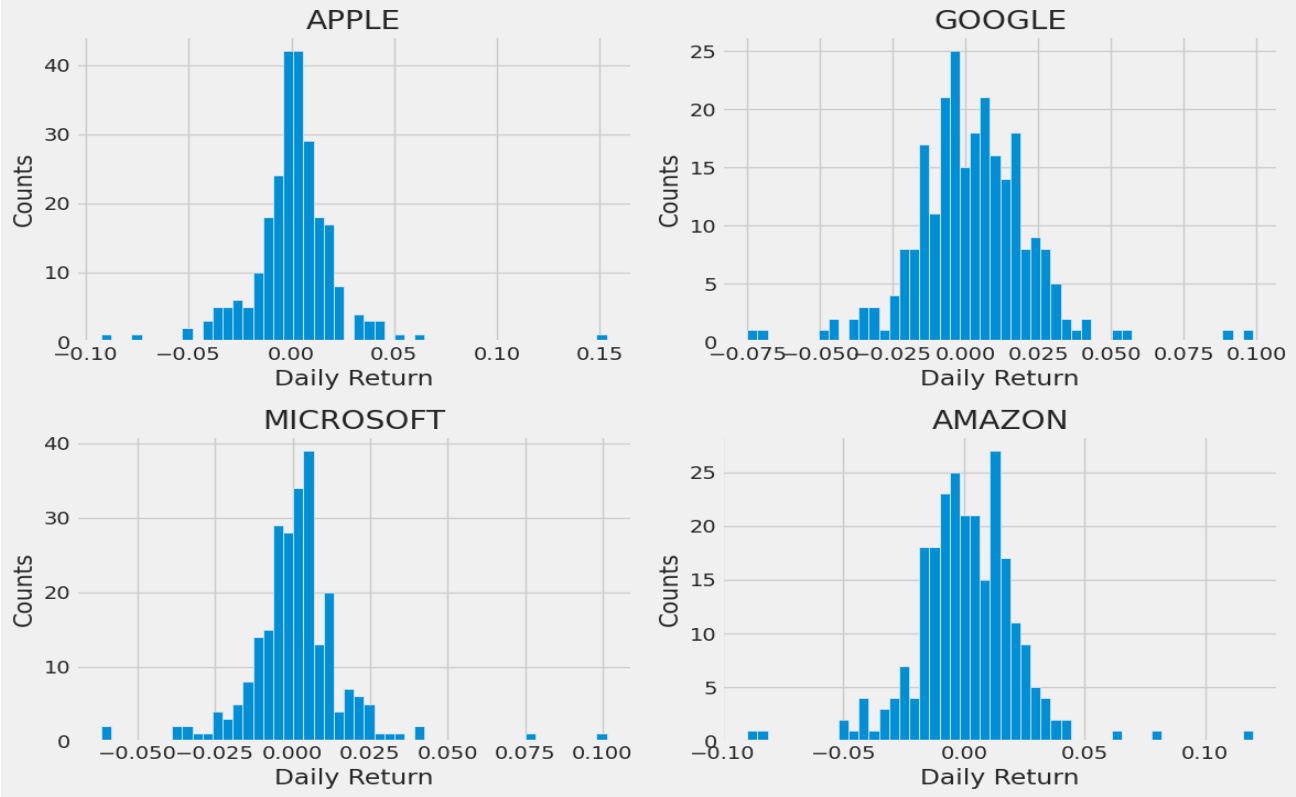
```
# Ensure same shape for RMSE calculation
y_test = y_test.reshape(-1, 1)


# Get the root mean squared error (RMSE)
rmse = np.sqrt(np.mean((predictions - y_test) ** 2))
print("Root Mean Squared Error (RMSE):", rmse)
```

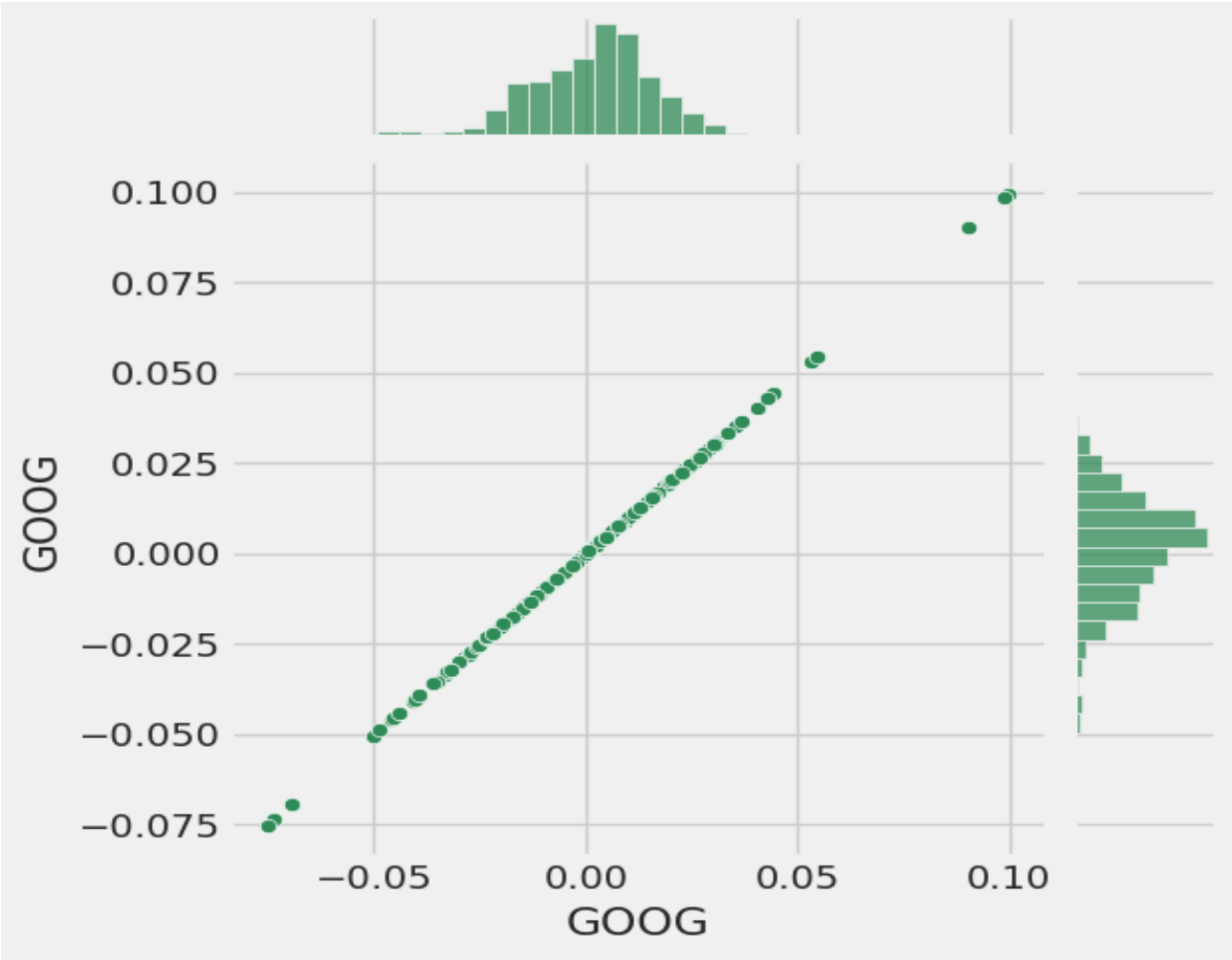


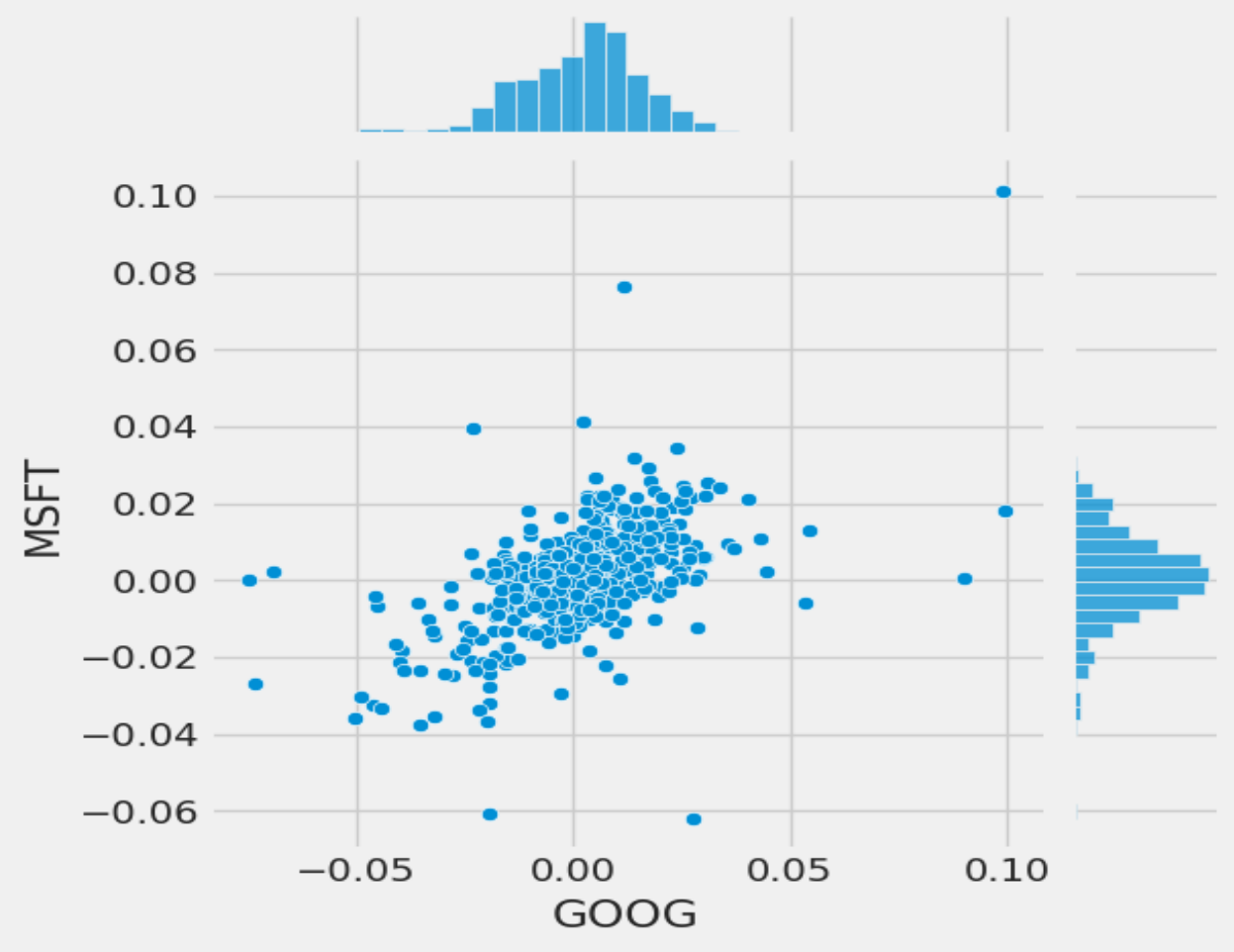


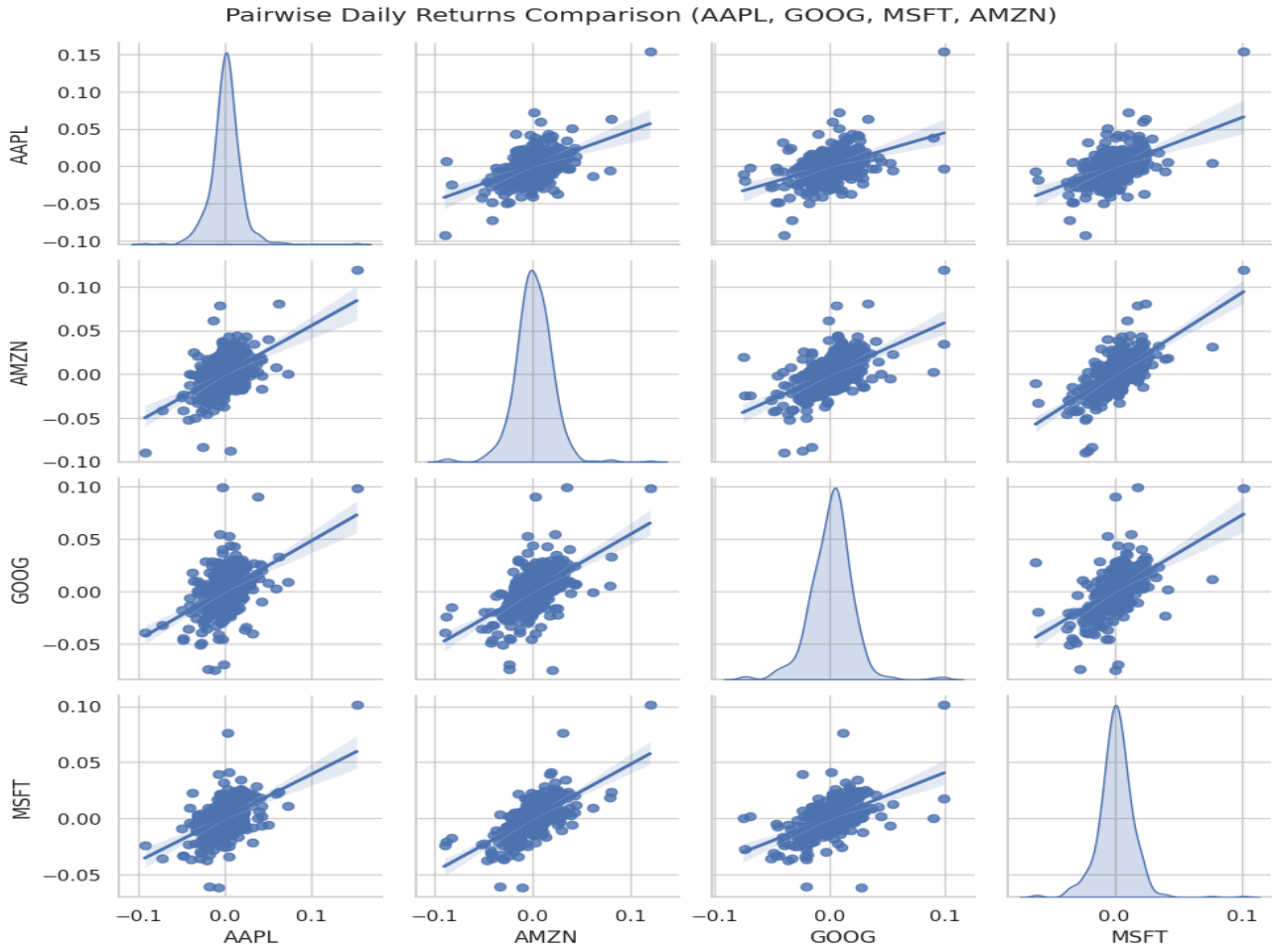

AAPL
GOOG
MSFT
AMZN
Price (USD)
Date
Price,Ticker
(Close, AAPL)
(Close, GOOG)
(Close, MSFT)
(Close, AMZN)
(MA for 10 days,)
(MA for 20 days,)
(MA for 50 days,)

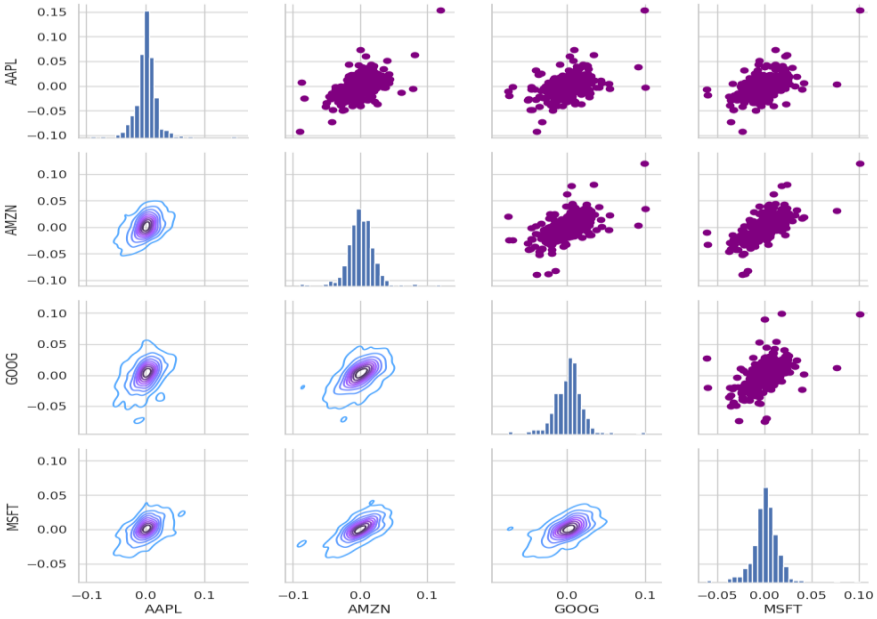

AAPL - Daily Return
GOOG - Daily Return
MSFT - Daily Return
AMZN - Daily Return
Daily Return
Daily Return (%)
Date

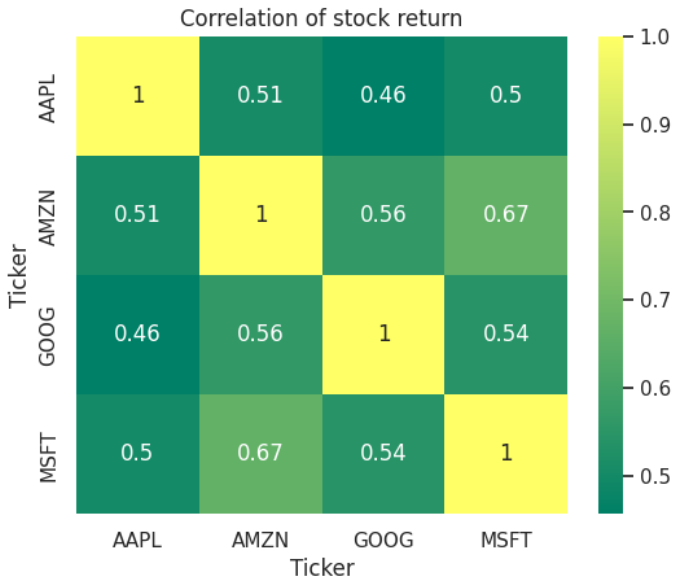

APPLE
GOOGLE
MICROSOFT
AMAZON
Counts
Daily Return

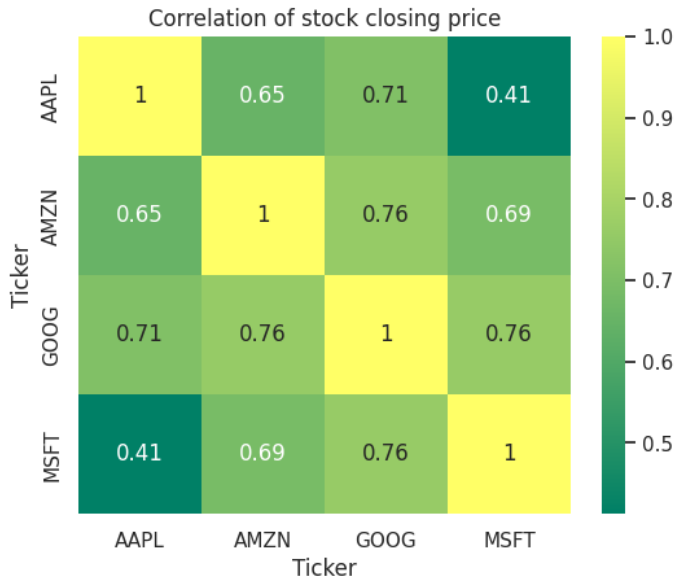

GOOG
GOOG


MSFT
GOOG
0.10
0.08
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
−0.05
0.00
0.05
0.10


Pairwise Daily Returns Comparison (AAPL, GOOG, MSFT, AMZN)
AAPL
AMZN
GOOG
MSFT
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.1
0.0
0.1


AAPL
AMZN
GOOG
MSFT


Correlation of stock return
Ticker
AAPL
AMZN
GOOG
MSFT
1 0.51 0.46 0.5
0.51 1 0.56 0.67
0.46 0.56 1 0.54
0.5 0.67 0.54 1


Correlation of stock closing price
Ticker
AAPL
AMZN
GOOG
MSFT
1 0.65 0.71 0.41
0.65 1 0.76 0.69
0.71 0.76 1 0.76
0.41 0.69 0.76 1

Close Price History
Close Price USD ($)
Date

❖ CONCLUSION

The **Stock Market Trend Traditions** project successfully demonstrates how data analytics and visualization techniques can be applied to understand and predict market behavior. By utilizing datasets from **Kaggle** and managing the workflow through **GitHub**, the project ensures reliability, scalability, and organized version control. The use of **Google Colab** and **Jupyter Notebook** provided a flexible environment for coding, analysis, and visualization, enabling efficient data handling and exploration.

Through systematic steps — from data collection to deployment — the project highlights the importance of structured analysis in financial forecasting. It transforms large volumes of stock data into meaningful insights that can support informed investment decisions. Interactive dashboards and graphical outputs make trend patterns easier to interpret, reflecting the real-world applications of data-driven decision-making in the stock market domain.

Overall, the project emphasizes the potential of modern data science tools in enhancing financial analysis. It not only strengthens understanding of stock market trends but also provides a foundation for future research and development in algorithmic trading and predictive analytics.

❖REFERENCE

Sl. No.	Components / Resources	Description / Purpose
1.	**Dataset Source**	Datasets collected from **Kaggle** and **GitHub**.
2.	**Development Platforms**	Coding done in **Jupyter Notebook** and **VS Code**.
3.	**Reference Codes**	Used **Kaggle notebooks** for reference.
4.	**Programming Language**	Implemented using **Python**.
5.	**Libraries Used**	**Pandas**, **NumPy**, **Matplotlib**, **Seaborn**, **Plotly**.
6.	**Data Storage**	Stored and managed in **GitHub repository**.
7.	**Analysis Output**	Generated graphs and trend visualizations.